CS781 Capstone Project: In-House Legal Intelligence System

Kartheek Dasari
261110029
kartheekd26@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

In real world, Law firms often deal with confidential information that cannot be shared with third-party services. To deal with this problem, a legal language system was developed to run locally and focus on Indian legal text.The work covers seven areas, starting with language model pretraining and including NER, rhetorical role prediction, judgment prediction, statute identification, summarization, and case retrieval. Foundation model is basically a 86.6M-parameter decoder-only Transformer built with RMSNorm, RoPE, and GQA. The model was trained from scratch on 300M Supreme Court tokens with a custom BPE tokenizer containing 32,000 tokens and then it wasadapted to each task using full or partial fine-tuning, LoRA, QLoRA, AdaLoRA, LoRMA, and distillation. Results vary across all phases with different methods dominating in each phase,In few phases LoRA outperformed full fine-tuning while training fewer parameters in other hand the distilled student improved on its teacher in Phases 4, 6, and 7 also results in few phases also show that task-specific choices, such as using a BiLSTM head or selecting the end of a document, can matter more than the adaptation method.

1 Competition Result

Codabench username: k_261110029 **Roll number:** 261110029

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	0.8343	47	6
2	Legal NER	0.5476	47	5
3	Rhetorical role prediction	0.5458	29	7
4	Court judgment prediction	0.7248	68	7
5	Legal statute identification	0.386	53	7
6	Abstractive summarization	0.8274	100	7
7	Prior Case Retrieval (PCR)	0.3810	72	7

Total valid Codabench submissions across all phases: 46
Date ranks were recorded: 8-11-2026
The per-phase minimum of four distinct submissions was met in every phase.

2 Introduction and Problem Description

Legal documents and communications can contain sensitive information, law firms may be unable to use third-party services to process them. To address this, we built a language model for Indian legal text that runs locally. We trained it from scratch, adapted it for seven tasks, and evaluated it across all seven phases on Codabench.

Tasks. Table 1 gives an overview of the seven phases. These tasks are continuation generation, NER with 12 entity types and exact character spans, prediction of 13 rhetorical roles in Corporate Law and IT, judgment outcome prediction, statute identification, abstractive summarization and retrieval of the top seven prior cases.

Table 1: The seven phases: input, output and primary metric.

Phase	Task	Input	Output	Primary metric
1	Legal LM pretraining	512-word prompt	128-word continuation	BERTScore F1
2	Legal NER	Legal document	Character spans + 12 labels	Strict Macro-F1
3	Rhetorical roles	Document sentences	One label per sentence	Macro-F1
4	Judgment prediction	Case text	Binary label	Macro-F1
5	Statute identification	Case text	Set of statute IDs	Macro-F1
6	Summarization	Legal document	Abstractive summary	BERTScore F1
7	Prior case retrieval	Query case + pool	Top-7 case IDs	Micro-F1@7

Constraints. The base model had to be trained from scratch, without pretrained weights and shoudl contain no of parameters count between 50M–300M parameters, use only the official data and run on free Kaggle GPUs, with checkpoints saved regularly. Every later phase after Phase-1 starts from Phase-1 checkpoint and required at least four clearly different experiments, including LoRMA and knowledge distillation. Each task contains specific challenges: NER required exact span matches, Macro-F1 was sensitive to rare labels, and BERTScore was used for language generation and summarization.

Research questions and scope. Six main questions were studied and explored in the project First, decoder-only model designs were compared: RoPE, mixture-of-experts with sandwich normalization, hybrid local and global attention, and RMSNorm with GQA and was selected as the base model. Second, transfer to token-, sentence-, document- and retrieval-level tasks was tested. Third, full and partial fine-tuning, LoRA, QLoRA, AdaLoRA, LoRMA, prompt tuning, prefix tuning and distillation were compared. The remaining parts were dealed with different ways of splitting and combining long judgments, different threshold settings and distillation into smaller models. An end to end pipeline was implemented starting from tokenization, pretraining, task heads,following adaptation methods, evaluation and Codabench submissions.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

Only the official dataset, `Exploration-Lab/CS-781-Capstone`, was used. Phase 1 has four data sources: Acts, High Courts, Supreme Court and Tribunals, on combined contain more than 4.5B

tokens. For the foundation model, 55,549 Supreme Court judgments were used. They were split 80:20 (seed 29) into 44,440 training documents and 11,109 validation documents, producing 300M training tokens and 3M validation tokens in blocks of 1,536 tokens (195,312 and 1,953 sequences). Phase 2 contains two labelled folds one with 4,120 and another one with 4,273 entities,also contains unlabelled test fold. Phase 3 contains 80 training documents with 16,845 sentences and 10 development documents with 2,142 sentences. Phase 4 has 37,387 training cases, 3,505 development cases and 1,517 test cases. Phase 5 has 42,750 training, 10,181 development and 13,019 test examples across 100 statutes. Phase 6 has 6,930 training documents, including 100 held-out documents, and 100 test documents. Phase 7 has 827 training queries with 4,320 candidates, 118 development queries with 1,023 candidates, and a test split with a separate candidate pool.

3.2 Corpus statistics that informed design

Phase 1 uses only Supreme Court judgments, so the training data mainly contains long legal decisions. In Phase 2, six entity types (DATE, PREC, AUTH, STAT, COURT and CASENO) make up 82% of all entities, and AUTH appears in only 18 of the 35 documents. Most entity spans are 10–15 characters long and do not overlap, which supports flat BIO tagging. In Phase 3, RatioOfTheDecision (4,301) and Fact (4,098) are the most common roles, while Issue (107) and PrecedentOverruled (3) are rare. Documents have a median of 9,446 tokens and the longest has 126,432 tokens, so chunking was needed. In Phase 4, the training labels are split 59:41, while the development set is balanced. The median case has 4,861 tokens, and 89–99% of cases exceed 1,024 tokens, which motivated chunking and methods that use the end of each document. In Phase 5, cases contain 3.7–3.9 statutes on average (maximum 31), about 21% of cases have one statute, and each statute has at least 272 positive training examples. In Phase 6, the median document length is 6,762 tokens and the median summary length is 1,382 tokens; with a context length of 1,536 and a summary budget of 256 tokens, only 1,279 document tokens can be used. In Phase 7, the number of relevant cases differs for each query, so returning only seven cases can miss relevant cases when a query has more than seven.

3.3 Preprocessing

We trained a 32,000-token BPE tokenizer from scratch , retaining information about the positions of words, punctuation, and whitespace. In all phases except Phase 2, NFKC normalization was applied, words split by line-break hyphens were repaired and repeated whitespace was combined and for Phase 2, the original character positions were kept because exact span matching was required, and each whitespace run was treated as one token with its original positions,converting the gold spans in this way gave a strict F1 of 0.994 and then Documents were processed in windows of 1,024 tokens with a stride of 768 and rredictions from overlapping windows are merged, aligned to word boundaries, and adjacent spans with the same label are also joined.

For Phase 3, sentences were joined using separator tokens and one label was predicted for each sentence. Three inputs were compared: the current sentence only, the current sentence with up to eight previous sentences (384 tokens), and the full document context. For Phase 4, long cases were split into chunks, encoded and combined, and representations from the end of a document were compared with those from multiple chunks. In Phase 5, statute IDs are mapped to a fixed set of labels and then represented as multi-hot vectors. Long cases are being split into non-overlapping chunks of 1,024 tokens, and selecting upto 4 chunks during training and evenly spaced chunks used during validation and prediction. For Phase 6, each document and its summary were joined using separator and end tokens, and the loss was calculated only on summary tokens. In Phase 7, for

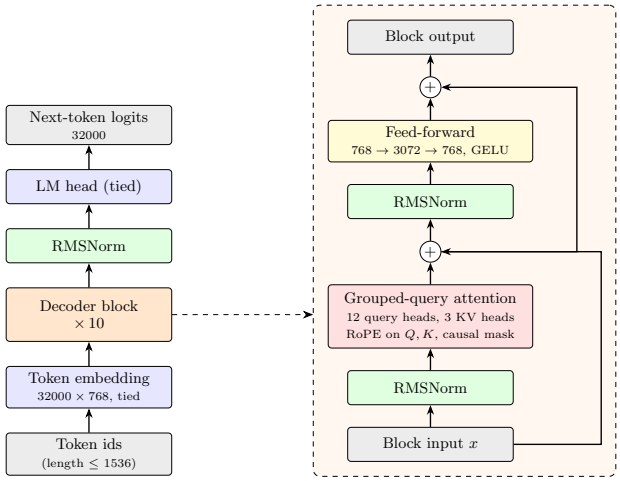


Figure 1: Phase-1 base model (v7): architecture and decoder block.

training one 1,024-token window is used which contains the beginning and end of each case, while validation and prediction used three evenly spaced windows and the hidden states are averaged and L2-normalized to create one embedding for each case.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

The final base model, v7, is a decoder-only Transformer with 10 layers, a width of 768 and a GELU feed-forward layer of size 3,072. It uses tied embeddings, RMSNorm, RoPE and GQA, with 12 query heads and 3 key-value heads, which makes the KV cache four times smaller and the model contains 86.6M parameters, a vocabulary of 32,000 tokens and a context length of 1,536. Its byte-level BPE tokenizer uses `<pad>`, `<unk>`, `<s>` and `</s>`.

The foundation model was trained on 195,312 blocks containing 300M tokens using next-token cross-entropy.and uses AdamW with $\beta = (0.9, 0.95)$, a learning rate of 3×10^{-4} and weight decay of 0.1 and then training used 500 warmup steps, cosine learning-rate decay, gradient clipping at 1.0, fp16 and gradient checkpointing and the batch size was 8 with 4 gradient accumulation steps, giving 32 blocks per update and then the model was evaluated every 500 steps and checkpoints were saved every 10 minutes and for cases like when Prompts longer than 1,152 tokens were truncated and uring generation, the model produced up to 384 tokens using temperature 0.7, top-p 0.95 and top-k 50 and generation stopped at `</s>` and was limited to 128 words, with no repetition penalty.

4.2 Transfer protocol (Phases 2–7)

Each phase starts by loading the Phase-1 v7 checkpoint and replacing its language-model head with a head for the target task and each phase was trained separately, as shown in Figure 2. The methods tested were full fine-tuning, partial fine-tuning of the last four blocks, rank-32 LoRA, rsLoRA, QLoRA, AdaLoRA, LoRMA, prompt tuning, prefix tuning and distillation. AdaLoRA, LoRMA and QLoRA were implemented from scratch. The best Phase 2 result came from LoRA with a BiLSTM head. LoRA performed best in Phase 3, distillation in Phases 4 and 7, and full fine-tuning in Phase 5 (Section 6). Phase 7 uses contrastive fine-tuning of a shared bi-encoder and

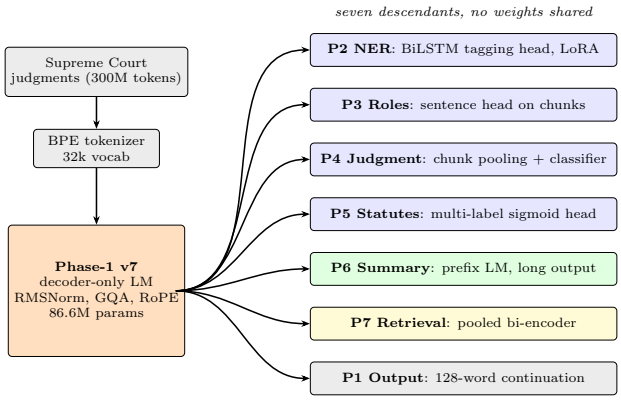


Figure 2: Shared Phase-1 backbone and separately adapted downstream models.

combines its scores with TF-IDF scores.

4.3 Phase 2 – Legal NER

NER uses token classification, and the original character spans are reconstructed using tokenizer offsets. The 12 entity types produce 25 BIO tags. Gold spans were mapped using the tokenizer offsets and predicted spans were rebuilt in the same way, which reached a strict F1 of 0.994. Each document was processed in 1,024-token windows with a stride of 768, scores from overlapping windows were combined, and unweighted cross-entropy loss was used. The final model combines v7, rank-32 LoRA and a BiLSTM head with hidden size 512, with only 9.2% of the parameters trainable. Viterbi decoding enforces valid BIO transitions. The flat tagging setup does not allow overlapping spans, and no thresholding or n-best decoding was used.

4.4 Phase 3 – Rhetorical role prediction

Documents were split into chunks of at most 1,024 tokens, with separator tokens placed after sentences. The hidden state at each separator represents its sentence, and up to eight previous sentences (384 tokens) serve as left context. A linear head predicts one of the 13 roles in a single pass. One model was trained jointly on Corporate Law and IT, without a CRF or class weights. The final model uses LoRA on v7.

4.5 Phase 4 – Court judgment prediction

Only the last 1,024 tokens of each case are kept to retain the decision section. The hidden state of a marker token is passed to a linear head that predicts one of two classes. Using the beginning of the document, both the beginning and end, and multiple chunks combined by mean, max or attention pooling were compared, and keeping only the end gave the best result. The final model is a five-block distilled student with 55.6M parameters, trained using an AdaLoRA teacher.

4.6 Phase 5 – Legal statute identification

Here up to four evenly spaced chunks of 1,024 tokens each are encoded, and their marker-token scores are combined by taking the maximum logit and a linear head predicts 100 independent labels using sigmoid outputs and binary cross-entropy loss, without class weights or label-wise attention

and the final model uses full fine-tuning of v7 and a threshold of 0.64, selected on the development set, to make sure that at least one statute is predicted for each case.

4.7 Phase 6 – Abstractive summarization

For summarization, the Phase-1 decoder is trained on the sequence `[document] <sep> [summary] <eos>`, with the loss calculated only on summary tokens. The final model uses LoRA and no encoder-decoder architecture. The context length is 1,536 tokens and the summary budget is 256 tokens, leaving room for up to 1,279 document tokens selected from four evenly spaced windows. Selection from the beginning, the end, both ends and important sentences was compared. During generation, summaries contain 8–256 tokens, with a repetition penalty of 1.05, 4-gram blocking and top-p 0.95.

4.8 Phase 7 – Prior Case Retrieval (PCR)

Retrieval uses a shared bi-encoder with the language-model head removed. Phase-1 hidden states are mean-pooled and L2-normalized to create case embeddings and for contrastive training uses temperature of 0.05, four hard negatives per query mined using TF-IDF, and a batch size of eight queries and cosine similarity is combined with a TF-IDF score, and the weight of each score is selected on the development set after normalization. The system returns the top seven cases and each case is represented by a 1,024-token window containing its beginning and end, and longer documents use three windows during evaluation.

5 Experiments and Implementation Details

5.1 Compute and training protocol

Experiments were run on free Kaggle GPUs. In Phase 1, checkpoints were saved every 10 minutes, and in later phases the best checkpoint of each run was saved. All phases used AdamW, fp16 and gradient clipping at 1.0. Phase 1 used 500 warmup steps, cosine learning-rate decay and weight decay of 0.1, while later phases used 5–10% warmup, linear decay and weight decay of 0.01. Dropout was set to 0.1, or 0.05 for LoRA. Class weights were not used in Phases 2–5 and each run used a fixed number of epochs without early stopping. The best checkpoint was selected using validation loss for Phases 1 and 6, strict Macro-F1 for Phase 2, Macro-F1 for Phases 3–5, and F1@7 for Phase 7.

Table 2: Hyperparameters of the final model in each phase.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Final model	v7	LoRA+BiLSTM	LoRA	KD student	full FT	LoRA	KD student
Parameters (M)	86.6	101.2	86.6	55.6	86.6	86.6	55.5
Context length	1536	1024	1024	1024	4×1024	1536	1024
Peak LR (head/body)	3e-4	1e-3/5e-4	1e-3/5e-4	1e-3/5e-5	1e-3/2e-5	1e-4	2e-5
Batch × accum.	8×4	2×8	2×8	2×8	1×16	2×8	8 queries
Epochs / steps	3	15	15	5	5	8	10
Dropout	0.1	0.1	0.1	0.1	0.1	0.1	0.1
Decoding / threshold	sampling[a]	Viterbi	argmax	argmax	0.64	sampling[b]	top-7

[a] temperature 0.7, top-p 0.95, top-k 50. [b] top-p 0.95, repetition penalty 1.05, 4-gram blocking.

5.2 Local evaluation

For Phase 1, 20% of the documents were used for validation, in total 3M tokens and Phase 2 used fold 2, while Phases 3–5 and 7 used their official development sets and for Phase 6, 100 held-out documents were evaluated using ROUGE, summary length and repetition rate; BERTScore was not used locally as it requires a pretrained model and local and Codabench rankings were mostly similar, but the scores differed. For example, Phase 2 scored 0.522 locally and 0.548 on Codabench, and Phase 5 distillation scored 0.378 locally and 0.307 on Codabench.

5.3 Ablations and controlled comparisons

Backbone. The best backbone depended on the task. With LoRA, Phase 3 scored 0.546 using v7, 0.540 using v5, 0.512 using v2 and 0.456 using v6. Phase 6 scored 0.827 with v7, 0.810 with v2 or v5, and 0.761 with v6. In Phase 5, full fine-tuning gave similar scores across backbones (0.379–0.386), as in Phase 7 (0.378–0.381). Phase 4 scored 0.723 with v6, 0.707 with v7 and 0.689 with v2. No single backbone was therefore best for every task.

Method. In Phase 2, LoRA scored 0.504, higher than full fine-tuning at 0.484, and adding a BiLSTM head increased the score to 0.548. Increasing the LoRA rank to 64 gave 0.502, which was not better than rank 32. In Phase 4, AdaLoRA scored 0.712, slightly above full fine-tuning at 0.707, while LoRMA scored 0.422. In Phase 5, full fine-tuning performed best at 0.386, followed by distillation (0.307), LoRMA (0.160) and AdaLoRA (0.030).

Distillation. The five-block student has 55.6M parameters. It improved the Phase 4 score to 0.725, compared with 0.712 for its AdaLoRA teacher and 0.707 for full fine-tuning and in Phase 7 it gave the best score of 0.3810. However, it performed worse in Phase 3 (0.474 compared with 0.546) and Phase 5 (0.307 compared with 0.386). Distillation can therefore reduce model size without reducing accuracy in some tasks, but its effect depends on the task.

6 Results

Table 3: Per-phase variant comparison (Codabench primary metric).

Phase	Variant	Metric
1	v7 (foundation model)	0.834
2	LoRA + BiLSTM	0.548
2	Full fine-tuning	0.484
2	Distillation	0.441
2	LoRMA	0.437
2	Partial fine-tuning	0.304
3	LoRA	0.546
3	LoRMA	0.495
3	Distillation	0.474
3	QLoRA	0.471
4	Distillation	0.725
4	AdaLoRA	0.712
4	Full fine-tuning	0.707
4	LoRMA	0.422
5	Full fine-tuning	0.386
5	Distillation	0.307
5	LoRMA	0.160
5	AdaLoRA	0.030
6	LoRA	0.827
6	Distillation	0.808
6	LoRMA	0.806
6	AdaLoRA	0.805
7	Distillation	0.381
7	Full fine-tuning	0.379
7	LoRMA	0.373
7	QLoRA	0.362

Phase 1 had about 6 valid submissions; only the best model (v7) is listed.

Which system won, and why. No single method performed best in every phase. LoRA worked well for the smaller token- and sentence-level tasks, while full fine-tuning worked better for Phase 5,

which predicts 100 independent labels. Distillation helped in Phase 4 because the smaller student learned useful representations from its teacher, but this improvement did not carry over to every task.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

- **P2 (NER).** AUTH appears in only 18 of 35 documents, and COURT, CASENO and STAT also need context to be identified correctly, which makes rare entities difficult to predict. Strict span matching also means that an error of even one character counts as both a false positive and a false negative.
- **P3 (roles).** Issue (107) and PrecedentOverruled (3) occur very rarely. Without class weights, these labels may not be learned well, which lowers Macro-F1, and the imbalance makes the common labels Fact and RatioOfTheDecision easier to predict.
- **P4 (judgment).** Only the last 1,024 tokens are kept, even though 89–99% of cases are longer. Reasoning about the outcome that appears earlier in a judgment may therefore be missed, and cases with similar facts but different outcomes may be confused.
- **P5 (statutes).** A single threshold of 0.64 may not work equally well for all 100 labels because their frequencies differ and using only four chunks may also miss statutes mentioned elsewhere in a case, which can reduce recall and this may partly explain the distillation score gap (0.378 vs. 0.307).
- **P6–P7.** In Phase 6, a summary may get a good ROUGE score but still miss the main decision or repeat the same points and in Phase 7, selecting only the beginning and end of a case may miss important information in the middle, and the top-7 limit prevents more than seven relevant cases from being returned and lexical scoring may also favour cases that use similar words rather than cases with similar legal reasoning.

7.2 What the metrics do not measure

Here the role of BERTScore and ROUGE is to check how similar an output is to a reference, but it cannot deal whether the legal content is correct and a fluent summary can still receive a good score even if it leaves out the court's decision or an important statute and also these metrics do not check whether citations are invented or whether a summary remains faithful to the judgment and strict span-F1 gives no credit when an entity has the correct label but an incorrect boundary, and Macro-F1 can hide poor results on rare labels also in Phase 7, ranking metrics do not measure the legal importance or precedential value of a case. A real system would therefore need lawyers to review its outputs.

7.3 Limitations and future work

The model was trained on 300M tokens of Supreme Court text, so the language of High Court and tribunal documents was not seen during pretraining. Long documents also have to be truncated or split into chunks, and no human evaluation of legal correctness was carried out. In future work, separate thresholds for each label should be tested in Phase 5. and for Phase 4, tail-only input should be compared with hierarchical chunk encoding on cases longer than 4,000 tokens. The other three data sources should be added to Phase-1 training and their effect on Phase 2 measured.

Finally, Phase 7 should be tested with different retrieval cut-offs, dense-only against hybrid scoring, and human judgments of case relevance.

7.4 Surprising findings

One surprising result was that LoRA (0.504) performed better than full fine-tuning (0.484) in Phase 2. Here no backbone was best across all tasks. AdaLoRA performed well as a teacher in Phase 4, with a score of 0.712, but scored only 0.030 in Phase 5 and the distilled student(kd) in Phase 4 performed better than its teacher, scoring 0.725 and these results show that the best adaptation method depends on the task, and that distillation is useful when the smaller student can retain the information needed for that task.

8 Conclusion

The results show that no single method is best for every phase and LoRA with a BiLSTM head gave the best result in Phase 2 (0.548), and LoRA gave the best results in Phase 3 (0.546) and Phase 6 (0.827). Distillation performed best in Phase 4 (0.725) and Phase 7 (0.381), while full fine-tuning performed best in Phase 5 (0.386).However Overall,choice of adaptation method and the way input documents are handled should depend on the task.

GitHub Repo Link: https://github.com/kartheek2159/llm_capstone_project.git

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: ChatGPT 6.0(free tier)

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	LLM?	Purpose
Understanding concepts / papers	Yes	Base concepts
Ideation / brainstorming	No	
Debugging small ancillary snippets	Yes	Small snippets
Plotting / visualization code	No	
Grammar / writing style	No	

Part	LLM?	Purpose
Report structure / section wording	No	
Designing model architecture	No	
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Core training / inference code	No	

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. The papers consulted are listed in the references section.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

The Phase-1 foundation model was designed and trained from scratch: four designs (RoPE, mixture-of-experts with sandwich normalization, hybrid local/global attention, and RMSNorm with GQA) were compared, and the 86.6M-parameter v7 model was selected. The BPE tokenizer and the data pipeline (1,536-token blocks of 300M Supreme Court tokens) were built. For later phases, the task heads and input handling were implemented, including offset-based token classification for NER, marker-token chunk pooling for statutes and a dense+TF-IDF bi-encoder for retrieval and peft techniques like AdaLoRA, LoRMA and QLoRA were implemented from scratch, and all methods were run under the same training setup.Corpus statistics guided chunking, tail-only truncation and the global threshold and some experiments did not work well.Here AdaLoRA scored 0.712 in Phase 4 but only 0.030 in Phase 5, suggesting its rank-pruning schedule may not suit a long-input multi-label task and LoRA rank 64 (0.502) did not beat rank 32 (0.504), and distillation underperformed its teacher in Phases 3 and 5. The best method therefore depends on the task and on how well the student learns from its teacher.

References

[1] A. Radford, K. Narasimhan, T. Salimans and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI Technical Report, 2018.

[2] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[3] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[4] I. Beltagy, M. E. Peters and A. Cohan, "Longformer: The long-document Transformer," arXiv:2004.05150, 2020.

[5] Z. Huang, W. Xu and K. Yu, "Bidirectional LSTM-CRF models for sequence tagging," arXiv:1508.01991, 2015.

[6] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[7] H. Bihany, S. Patel and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," arXiv:2506.07621, 2025.

[8] G. Hinton, O. Vinyals and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[9] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[10] Q. Zhang et al., "Adaptive budget allocation for parameter-efficient fine-tuning," in *Proc. ICLR*, 2023.